

Chapter 17. Asynchronous Language Features

C# provides language-level support for using and implementing asynchronous methods. Asynchronous APIs are often the most efficient way to use certain services. For example, most I/O is handled asynchronously inside the OS kernel, because most peripherals, such as disk controllers or network adapters, are able to do the majority of their work autonomously. They need the CPU to be involved only at the start and end of each operation.

Although many of the services offered by operating systems are intrinsically asynchronous, developers often choose to use them through synchronous APIs (i.e., ones that do not return until the work is complete). This can waste resources, because they block the thread until the I/O completes. Threads have overheads, and if you're aiming to get the best performance in a highly concurrent application (e.g., a web app serving large numbers of users), it's usually best to have a relatively small number of OS threads. Ideally, your application would have no more OS threads than you have hardware threads, but that's optimal only if you can ensure that threads only ever block when there's no outstanding work for them to do. ([Chapter 16](#) described the difference between OS threads and hardware threads.) The more threads that get blocked inside synchronous API calls, the more threads you'll need to handle your workload, reducing efficiency. In performance-sensitive code, asynchronous APIs are useful, because instead of wasting resources by forcing a thread to sit and wait for I/O to complete, a thread can kick off the work and then do something else productive in the meantime.

The problem with asynchronous APIs is that they can be significantly more complex to use than synchronous ones, particularly if you need to coordinate multiple related operations and deal with errors. This was often why developers chose the less efficient synchronous alternatives back

in the days before any mainstream programming languages provided built-in support. In 2012, C# and Visual Basic both brought language-level support out of the research labs, and since then many other popular languages have added analogous features (most notably JavaScript, which acquired a very similar-looking syntax in 2016). The asynchronous features in C# make it possible to write code that uses efficient asynchronous APIs while retaining most of the simplicity of code that uses simpler synchronous APIs.

These language features are also useful in some scenarios in which maximizing throughput is not the primary performance goal. With client-side code, it's important to avoid blocking the UI thread to maintain responsiveness, and asynchronous APIs provide one way to do that. The language support for asynchronous code can handle thread affinity issues, which greatly simplifies the job of writing highly responsive UI code.

Asynchronous Keywords: `async` and `await`

C# presents its support for asynchronous code through two keywords: `async` and `await`. Neither of these is meant to be used on its own. You put the `async` keyword in a method's declaration, and this tells the compiler that you intend to use asynchronous features in the method. If this keyword is not present, you are not allowed to use the `await` keyword.

This is arguably redundant—the compiler produces an error if you attempt to use `await` without `async`. If it knows when a method's body is trying to use asynchronous features, why do we need to tell it explicitly? There are two reasons. First, as you'll see, these features radically change the behavior of the code the compiler generates, so it's useful for anyone reading the code to see a clear indication that the method behaves asynchronously. Second, `await` wasn't always a keyword in C#, so developers were once free to use it as an identifier. Perhaps Microsoft could have designed the grammar for `await` so that it acts as a keyword only in very specific contexts, enabling you to continue to use it as an identifier in all other scenarios, but the C# team decided to take a slightly more coarse-grained approach: you cannot use `await` as an identifier inside an `async` method, but it's a valid identifier anywhere else.

NOTE

The `async` keyword does not change the signature of the method. It determines how the method is compiled, not how it is used.

The program entry point is a special case. If you use top-level statements to avoid having to declare `Main` explicitly, there's no place to put the `async` keyword or a return type, so this is the one case where the compiler deduces whether a method is asynchronous from whether you use `await`.

So the `async` keyword simply declares your intention to use the `await` keyword. (While you mustn't use `await` without `async` anywhere other than in top-level statements, it's not an error to apply the `async` keyword to a method that doesn't use `await`. However, it would serve no purpose, so the compiler will generate a warning if you do this.) [Example 17-1](#) shows a fairly typical example. This uses the `HttpClient` class to request just the headers for a particular resource (using the standard `HEAD` verb that the HTTP protocol defines for this purpose). It then displays the results in a UI control—this method is part of the codebehind for a UI that includes a `TextBox` named `headerListTextBox`.

Example 17-1. Using `async` and `await` when fetching HTTP headers

```
// Note: as you'll see later, async methods usually should not be void
private async void FetchAndShowHeaders(string url, IHttpClientFactory cf)
{
    using (HttpClient w = cf.CreateClient())
    {
        var req = new HttpRequestMessage(HttpMethod.Head, url);
        HttpResponseMessage response =
            await w.SendAsync(req, HttpCompletionOption.ResponseHeadersRead);

        headerListTextBox.Text = response.Headers.ToString();
    }
}
```

This code contains a single `await` expression, shown in bold. You use the `await` keyword in an expression that may take some time to produce a

result, and it indicates that the remainder of the method should not execute until that operation is complete. This sounds a lot like what a blocking, synchronous API does, but the difference is that an `await` expression does not block the thread. This code is not quite what it seems.

The `HttpClient` class's `SendAsync` method returns a `Task<HttpResponseMessage>`, and you might be wondering why we wouldn't just use its `Result` property. As you saw in [Chapter 16](#), if the task is not complete, this property blocks the thread until the result is available (or the task fails, in which case it will throw an exception instead). However, this is a dangerous thing to do in a UI application: if you block the UI thread by trying to read the `Result` of an incomplete task, you will prevent progress of any operations that need to run on that thread. Since a lot of the work that UI applications do needs to happen on the UI thread, blocking that thread in this way more or less guarantees that deadlock will occur sooner or later, causing the application to freeze. So don't do that!

Although the `await` expression in [Example 17-1](#) does something that is logically similar to reading `Result`, it works very differently. If the task's result is not available immediately, the `await` keyword does not make the thread wait, despite what its name suggests. Instead, it causes the containing method to return. You can use a debugger to verify that `FetchAndShowHeaders` returns immediately. For example, if I call that method from the button click event handler shown in [Example 17-2](#), I can put a breakpoint on the `Debug.WriteLine` call in that handler and another breakpoint on the code in [Example 17-1](#) that will update the `headerListTextBox.Text` property.

Example 17-2. Calling the asynchronous method

```
private void fetchHeadersButton_Click(object sender, RoutedEventArgs e)
{
    FetchAndShowHeaders("https://endjin.com/", this.clientFactory);
    Debug.WriteLine("Method returned");
}
```

Running this in the debugger, I find that the code hits the breakpoint on the last statement of [Example 17-2](#) before it hits the breakpoint on the fi-

nal statement of [Example 17-1](#). In other words, the section of [Example 17-1](#) that follows the `await` expression runs *after* the method has returned to its caller. Evidently, the compiler is somehow arranging for the remainder of the method to be run via a callback that occurs once the asynchronous operation completes.

NOTE

Visual Studio's debugger plays some tricks when you debug asynchronous methods to enable you to step through them as though they were normal methods. This is usually helpful, but it can sometimes conceal the true nature of execution. The debugging steps I just described were contrived to defeat Visual Studio's attempts to be clever and instead to reveal what is really happening.

Notice that the code in [Example 17-1](#) expects to run on the UI thread because it modifies the text box's `Text` property toward the end. Asynchronous APIs do not necessarily guarantee to notify you of completion on the same thread on which you started the work—in fact, most won't. Despite this, [Example 17-1](#) works as intended, so as well as converting half of the method to a callback, the `await` keyword is handling thread affinity issues for us.

The C# compiler evidently performs some major surgery on your code each time you use the `await` keyword. In older versions of C#, if you wanted to use this asynchronous API and then update the UI, you would need to have written something like [Example 17-3](#). This uses a technique I showed in [Chapter 16](#): it sets up a continuation for the task returned by `SendAsync`, using a `TaskScheduler` to ensure that the continuation's body runs on the UI thread.

Example 17-3. Manual asynchronous coding

```
private void OldSchoolFetchHeaders(string url, IHttpClientFactory cf)
{
    HttpClient w = cf.CreateClient();
    var req = new HttpRequestMessage(HttpMethod.Head, url);

    var uiScheduler = TaskScheduler.FromCurrentSynchronizationContext();
    w.SendAsync(req, HttpCompletionOption.ResponseHeadersRead)
```

```

        .ContinueWith(sendTask =>
        {
            try
            {
                HttpResponseMessage response = sendTask.Result;
                headerListTextBox.Text = response.Headers.ToString();
            }
            finally
            {
                w.Dispose();
            }
        },
        uiScheduler);
    }

```

This is a reasonable way to use the TPL directly, and it has a similar effect to [Example 17-1](#), although it's not an exact representation of how the C# compiler transforms the code. As I'll show later, `await` uses a pattern that is supported by, but does not require, `Task` or `Task<T>`. It also generates code that handles early completion (where the task has already finished by the time you're ready to wait for it) far more efficiently than [Example 17-3](#). But before I show the details of what the compiler does, I want to illustrate some of the problems it solves for you, which is best done by showing the kind of code you might have written back before this language feature existed.

My current example is pretty simple, because it involves only one asynchronous operation, but aside from the two steps I've already discussed—setting up some kind of completion callback and ensuring that it runs on the correct thread—I've also had to deal with the `using` statement that was in [Example 17-1](#). [Example 17-3](#) can't use the `using` keyword, because we want to dispose the `HttpClient` object only after we've finished with it.¹ Calling `Dispose` shortly before the outer method returns would not work, because we need to be able to use the object when the continuation runs, and that will typically happen a fair bit later. So I need to create the object in one method (the outer one) and then dispose of it in a different method (the nested one). And because I'm calling `Dispose` by hand, it's now my problem to deal with exceptions, so I've had to wrap all of the code I moved into the callback with a `try` block and call `Dispose` in a `finally` block. (In fact, I've not even done a comprehensive job. In the unlikely event that either the `HttpRequestMessage` constructor or the

call that retrieves the task scheduler were to throw an exception, the `HttpClient` would not get disposed. I'm handling only the case where the HTTP operation itself fails.)

[Example 17-3](#) has used a task scheduler to arrange for the continuation to run via the `SynchronizationContext` that was current when the work started. This ensures that the callback occurs on the correct thread to update the UI. The `await` keyword can take care of that for us.

Execution and Synchronization Contexts

When your program's execution reaches an `await` expression for an operation that doesn't complete immediately, the code generated for that `await` will ensure that the current execution context has been captured. (It might not have to do much—if this is not the first `await` to block in this method, and if the context hasn't changed since, it will have been captured already.) When the asynchronous operation completes, the remainder of your method will be executed through the execution context.²

As I described in [Chapter 16](#), the execution context handles certain contextual information that needs to flow when one method invokes another (even when it does so indirectly). But there's another kind of context that we may be interested in, particularly when writing UI code: the synchronization context (which was also described in [Chapter 16](#)).

While all `await` expressions capture the execution context, the decision of whether to flow synchronization context as well is controlled by the type being awaited. If you `await` for a `Task`, the synchronization context will also be captured by default. Tasks are not the only thing you can `await`, and I'll describe how types can support `await` in the section [“The `await` Pattern”](#).

Sometimes, you might want to avoid getting the synchronization context involved. If you want to perform asynchronous work starting from a UI thread, but you have no particular need to remain on that thread, scheduling every continuation through the synchronization context is unnecessary overhead. If the asynchronous operation is a `Task` or `Task<T>` (or the equivalent value types, `ValueTask` or `ValueTask<T>`), you can declare that you don't want this by calling the `ConfigureAwait` method

passing `false`. This returns a different representation of the asynchronous operation, and if you `await` that instead of the original task, it will ignore the current `SynchronizationContext` if there is one. (There's no equivalent mechanism for opting out of the execution context.)

[Example 17-4](#) shows how to use this.

Example 17-4. `ConfigureAwait`

```
private async void OnFetchButtonClick(object sender, RoutedEventArgs e)
{
    using (HttpClient w = this.clientFactory.CreateClient())
    using (Stream f = File.Create(fileTextBox.Text))
    {
        Task<Stream> getStreamTask = w.GetStreamAsync(urlTextBox.Text);
        Stream getStream = await getStreamTask.ConfigureAwait(false);

        Task copyTask = getStream.CopyToAsync(f);
        await copyTask.ConfigureAwait(ConfigureAwaitOptions.None);
    }
}
```

This code is a click handler for a button, so it initially runs on a UI thread. It retrieves the `Text` property from a couple of text boxes. Then it kicks off some asynchronous work—fetching the content for a URL and copying the data into a file. It does not use any UI elements after fetching those two `Text` properties, so it doesn't matter if the remainder of the method runs on some separate thread. By passing `false` to `ConfigureAwait` and waiting on the value it returns, we are telling the TPL that we are happy for it to use whatever thread is convenient to notify us of completion, which in this case will most likely be a thread pool thread. This will enable the work to complete more efficiently and more quickly, because it avoids getting the UI thread involved unnecessarily after each `await`.

You might have spotted that the final call to `ConfigureAwait` in [Example 17-4](#) doesn't pass `false`. .NET 8.0 added a new overload taking a `ConfigureAwaitOptions`. This enumeration type's `None` and `ContinueOnCapturedContext` members provide the same behavior as `false` and `true`, and it also provides access to two new capabilities. We'll look at one of these, `SuppressThrowing`, in [“Error Handling”](#). The

other, `ForceYielding`, disables an optimization. Normally, in cases where the task has already completed (described in more detail in [“The await Pattern”](#)), `await` just allows the rest of the method to proceed immediately, but `ForceYielding` causes it to act as though the task had not completed. This typically causes the remainder of the method to run on a thread pool thread when it would otherwise have run on the caller’s thread. This might be useful if you know that after the `await`, you will be going on to perform CPU-intensive work, or to call some slow API that does not offer an asynchronous form, and you don’t want that to tie up the caller’s thread. `ConfigureAwaitOptions` is a flags-style enumeration (with `None` having the value 0) so you can ask for combinations of these behaviors.

Not all asynchronous APIs return `Task` or `Task<T>`. For example, various asynchronous APIs introduced to Windows as part of UWP (an API for building desktop and tablet applications) return an `IAsyncOperation<T>` instead of `Task<T>`. This is because UWP is not .NET-specific, and it has its own runtime-independent representation for asynchronous operations that can also be used from C++ and JavaScript. This interface is conceptually similar to TPL tasks, and it supports the await pattern, meaning you can use `await` with these APIs. However, it does not provide `ConfigureAwait`. If you want to do something similar to [Example 17-4](#) with one of these APIs, you can use the `AsTask` extension method that wraps an `IAsyncOperation<T>` as a `Task<T>`, and you can call `ConfigureAwait` on that task instead.

TIP

If you are writing libraries, then in most cases you should call `ConfigureAwait(false)` anywhere you use `await`. This is because continuing via the synchronization context can be expensive, and in some cases it can introduce the possibility of deadlock occurring. The only exceptions are when you are doing something that positively requires the synchronization context to be preserved, or you know for certain that your library will only ever be used in application frameworks that do not set up a synchronization context. (E.g., ASP.NET Core applications do not use synchronization contexts, so it generally doesn’t matter whether or not you call `ConfigureAwait(false)` in those.)

[Example 17-1](#) contained just one `await` expression, and even that turned out to be fairly complex to reproduce with classic TPL programming.

[Example 17-4](#) contains two, and achieving equivalent behavior without the aid of the `await` keyword would require rather more code, because exceptions could occur before the first `await`, after the second, or between, and we'd need to call `Dispose` on the `HttpClient` and `Stream` in any of those cases (as well as in the case where no exception is thrown). However, things can get considerably more complex than that once flow control gets involved.

Multiple Operations and Loops

Suppose that instead of fetching headers, or just copying the HTTP response body to a file, I wanted to process the data in the body. If the body is large, retrieving it is an operation that could require multiple, slow steps. [Example 17-5](#) fetches a web page gradually.

Example 17-5. Multiple asynchronous operations

```
private async void FetchAndShowBody(string url, IHttpClientFactory cf)
{
    using (HttpClient w = cf.CreateClient())
    {
        Stream body = await w.GetStreamAsync(url);
        using (var bodyTextReader = new StreamReader(body))
        {
            while (!bodyTextReader.EndOfStream)
            {
                string? line = await bodyTextReader.ReadLineAsync();
                bodyTextBox.AppendText(line);
                bodyTextBox.AppendText(Environment.NewLine);
                await Task.Delay(TimeSpan.FromMilliseconds(10));
            }
        }
    }
}
```

This now contains three `await` expressions. The first kicks off an HTTP GET request, and that operation will complete when we get the first part of the response, but the response might not be complete yet—there may

be several megabytes of content to come. This code presumes that the content will be text, so it wraps the `Stream` object that comes back in a `StreamReader`, which presents the bytes in a stream as text.³ It then uses that wrapper's asynchronous `ReadLineAsync` method to read text a line at a time from the response. Reading the first line may take a while because it will need to wait for the response to arrive over the network from the server, but the next few calls to this method will probably complete immediately, because each network packet we receive will typically contain multiple lines. But if the code can read faster than data arrives over the network, eventually it will have consumed all the lines that appeared in the first packet, and it will then take a while before the next line becomes available. So the calls to `ReadLineAsync` will return some tasks that are slow and some that complete immediately. The third asynchronous operation is a call to `Task.Delay`. I've added this to slow things down so that I can see the data arriving gradually in the UI. `Task.Delay` returns a `Task` that completes after the specified delay, so this provides an asynchronous equivalent to `Thread.Sleep`. (`Thread.Sleep` blocks the calling thread, but `await Task.Delay` introduces a delay without blocking the thread.)

NOTE

I've put each `await` expression in a separate statement, but this is not a requirement. It's perfectly legal to write expressions of the form `(await t1) + (await t2)`. (You can omit the parentheses if you like, because `await` has higher precedence than addition; I prefer the visual emphasis they provide here.)

I'm not going to show you the complete pre-`async` equivalent of [Example 17-5](#), because it would be enormous, but I'll describe some of the problems. First, we've got a loop with a body that contains two `await` blocks. To produce something equivalent with `Task` and callbacks means building your own loop constructs, because the code for the loop ends up being split across three methods: the one that starts the loop running (which would be the nested method acting as the continuation callback for `GetStreamAsync`) and the two callbacks that handle the completion of `ReadLineAsync` and `Task.Delay`. You can solve this by having a local method that starts a new iteration and calling that from two places: the point at which you want to start the loop and again in the `Task.Delay`

continuation to kick off the next iteration. [Example 17-6](#) shows this technique, but it illustrates just one aspect of what we’re expecting the compiler to do for us; it is not a complete alternative to [Example 17-5](#).

Example 17-6. An incomplete manual asynchronous loop

```
private void IncompleteOldSchoolFetchAndShowBody(
    string url, IHttpClientFactory cf)
{
    HttpClient w = cf.CreateClient();
    var uiScheduler = TaskScheduler.FromCurrentSynchronizationContext();
    w.GetStreamAsync(url).ContinueWith(getStreamTask =>
    {
        Stream body = getStreamTask.Result;
        var bodyTextReader = new StreamReader(body);

        StartNextIteration();

        void StartNextIteration()
        {
            if (!bodyTextReader.EndOfStream)
            {
                bodyTextReader.ReadLineAsync().ContinueWith(readLineTask =>
                {
                    string? line = readLineTask.Result;

                    bodyTextBox.AppendText(line);
                    bodyTextBox.AppendText(Environment.NewLine);

                    Task.Delay(TimeSpan.FromMilliseconds(10))
                        .ContinueWith(
                            _ => StartNextIteration(), uiScheduler);
                },
                uiScheduler);
            }
        };
    },
    uiScheduler);
}
```

This code works after a fashion, but it doesn’t even attempt to dispose any of the resources it uses. There are several places in which failure could occur, so we can’t just put a single `using` block or `try/finally` pair in

to clean things up. And even without that additional complication, the code is barely recognizable—it's not obvious that this is attempting to perform the same basic operations as [Example 17-5](#). With proper error handling, it would be completely unreadable. In practice, it would probably be easier to take a different approach entirely, writing a class that implements a state machine to keep track of where the work has gotten to. That will probably make it easier to produce code that operates correctly, but it's not going to make it any easier for someone reading your code to understand that what they're looking at is really little more than a loop at heart.

No wonder so many developers used to prefer synchronous APIs. But C# lets us write asynchronous code that has almost exactly the same structure as the synchronous equivalent, giving us all of the performance and responsiveness benefits of asynchronous code without the pain. That's the main benefit of `async` and `await` in a nutshell.

Consuming and producing asynchronous sequences

[Example 17-5](#) showed a `while` loop, and as you'd expect, you're free to use other kinds of loops such as `for` and `foreach` in `async` methods. However, `foreach` can introduce a subtle problem: What happens if the collection you iterate over needs to perform slow operations? This doesn't arise for collection types such as arrays or `HashSet<T>`, where all the collection's items are already in memory, but what about the `IEnumerable<string>` returned by `File.ReadLines`? That's an obvious candidate for asynchronous operation, but in practice, it will just block your thread each time it needs to wait for more data to arrive from storage. And that's because the pattern expected by `foreach` doesn't support asynchronous operation. The heart of the problem is the method `foreach` will call to move to the next item—it expects the enumerator (often, but not always an implementation of `IEnumerator<T>`) to provide a `MoveNext` method with a signature like the one shown in [Example 17-7](#).

Example 17-7. The non-async-friendly `IEnumerator.MoveNext`

```
bool MoveNext();
```

If more items are forthcoming but are not yet available, collections have no choice but to block the thread, not returning from `MoveNext` until the data arrives. Fortunately, C# recognizes a variation on this pattern. The runtime libraries define a pair of types,⁴ shown in [Example 17-8](#) (first introduced in [Chapter 5](#)), that embody this newer pattern. As with the synchronous `IEnumerable<T>`, `foreach` doesn't strictly require these exact types. Anything offering members of the same signature will work.

Example 17-8. `IAsyncEnumerable<T>` and `IAsyncEnumerator<T>`

```
public interface IAsyncEnumerable<out T>
{
    IAsyncEnumerator<T> GetAsyncEnumerator(
        CancellationToken cancellationToken = default);
}

public interface IAsyncEnumerator<out T> : IAsyncDisposable
{
    T Current { get; }

    ValueTask<bool> MoveNextAsync();
}
```

Conceptually this is identical to the synchronous pattern: an asynchronous `foreach` will ask the collection object for an enumerator and will repeatedly ask it to advance to the next item, executing the loop body with the value returned by `Current` each time until the enumerator indicates that there are no more items. The main difference is that the synchronous `MoveNext` has been replaced by `MoveNextAsync`, which returns an awaitable `ValueTask<T>`. (The `IAsyncEnumerable<T>` interface also provides support for passing in a cancellation token. An asynchronous `foreach` won't use that itself directly, but you can use this indirectly through the `WithCancellation` extension method for `IAsyncEnumerable<T>`.)

To consume an enumerable source that implements this pattern, you must put the `await` keyword in front of the `foreach`. C# can also help you to implement this pattern: [Chapter 5](#) showed how you can use the `yield` keyword in an *iterator* method to implement `IEnumerable<T>`, but you can also return an `IAsyncEnumerable<T>`. [Example 17-9](#) shows

both implementation and consumption of `IAsyncEnumerable<T>` in action.

Example 17-9. Consuming and producing asynchronous enumerables

```
await foreach (string line in ReadLinesAsync(args[0]))
{
    Console.WriteLine(line);
}

static async IAsyncEnumerable<string> ReadLinesAsync(string path)
{
    using (var bodyTextReader = new StreamReader(path))
    {
        while (!bodyTextReader.EndOfStream)
        {
            string? line = await bodyTextReader.ReadLineAsync();
            if (line is not null) { yield return line; }
        }
    }
}
```

Since this language support makes creating and using `IAsyncEnumerable<T>` very similar to working with `IEnumerable<T>`, you might be wondering whether there are asynchronous versions of the various LINQ operators described in [Chapter 10](#). Unlike LINQ to Objects, `IAsyncEnumerable<T>` implementations are not in the parts of the runtime libraries built into .NET or .NET Standard, but a suitable NuGet package is available. If you add a reference to the `System.Linq.Async` package, the usual `using System.Linq;` declaration will make all the LINQ operators available on `IAsyncEnumerable<T>` expressions.

While we're looking at asynchronous equivalents of widely implemented types, we should look at `IAsyncDisposable`.

Asynchronous disposal

As [Chapter 7](#) described, the `IDisposable` interface is implemented by types that need to perform some sort of cleanup promptly, such as closing an open handle, and there is language support in the form of `using` statements. But what if the cleanup involves potentially slow work, such

as flushing data out to disk? .NET and .NET Standard 2.1 provide the `IAsyncDisposable` interface for this scenario. As [Example 17-10](#) shows, you can put the `await` keyword in front of a `using` statement to consume an asynchronously disposable resource. (You can also put `await` in front of a `using` declaration.)

Example 17-10. Consuming and implementing `IAsyncDisposable`

```
await using (DiagnosticWriter w = new(@"c:\temp\log.txt"))
{
    await w.LogAsync("Test");
}

class DiagnosticWriter : IAsyncDisposable
{
    private StreamWriter? _sw;

    public DiagnosticWriter(string path)
    {
        _sw = new StreamWriter(path);
    }

    public Task LogAsync(string message)
    {
        ObjectDisposedException.ThrowIf(_sw is null, nameof(DiagnosticWriter));
        return _sw.WriteLineAsync(message);
    }

    public async ValueTask DisposeAsync()
    {
        if (_sw is not null)
        {
            await LogAsync("Done");
            await _sw.DisposeAsync();
            _sw = null;
        }
    }
}
```

NOTE

Although the `await` keyword appears in front of the `using` statement, the potentially slow operation that it awaits happens when execution leaves the `using` statement's block. This is unavoidable since `using` statements and declarations effectively hide the call to `Dispose`.

[Example 17-10](#) also shows how to implement `IAsyncDisposable`.

Whereas the synchronous `IDisposable` defines a single `Dispose` method, its asynchronous counterpart defines a single `DisposeAsync` method that returns a `ValueTask`. This enables us to annotate the method with `async`. An `await using` statement will ensure that the task returned by `DisposeAsync` completes at the end of its block before execution continues. You may have noticed that we've used a few different return types for `async` methods. Iterators are a special case, just as they are in synchronous code, but what about these methods that return various task types?

Returning a Task

Any method that uses `await` could itself run slowly, so as well as being able to call asynchronous APIs, you will usually also want to present an asynchronous public face. The C# compiler enables methods marked with the `async` keyword to return an object that represents the asynchronous work in progress. Instead of returning `void`, you can return a `Task`, or you can return a `Task<T>`, where `T` is any type. This enables callers to discover the status of the work your method performs, it provides the opportunity to attach continuations, and if you use `Task<T>`, it offers a way to get the result. Alternatively, you can return the value type equivalents, `ValueTask` and `ValueTask<T>`. Returning any of these means that if your method is called from another `async` method, it can use `await` to wait for your method to complete and, if applicable, to collect its result.

Returning a task is almost always preferable to `void` when using `async` because with a `void` return type, there's no way for callers to know when your method has really finished, or to discover when it throws an exception. (Asynchronous methods can continue to run after returning—in fact, that's the whole point—so by the time you throw an exception, the

original caller will probably not be on the stack.) By returning a task object, you provide the compiler with a way to make exceptions available and, where applicable, a way to provide a result.

Returning a task is so trivially easy that there's very little reason not to. To modify the method in [Example 17-5](#) to return a task, I only need to make a single change. I make the return type `Task` instead of `void`, as shown in [Example 17-11](#), and the rest of the code can remain exactly the same.

Example 17-11. Returning a `Task`

```
private async Task FetchAndShowBody(string url, IHttpClientFactory cf)
// ...as before
```

The compiler automatically generates the code required to produce a `Task` object (or a `ValueTask`, if you use that as your return type) and set it into a completed or faulted state when the method either returns or throws an exception. A return type of `Task` is the asynchronous equivalent of `void`, since the `Task` produces no result when it completes (which is why we don't need to add a `return` statement to this method even though it now has a return type of `Task`). And if you want to return a result from your task, that's also easy. Make the return type `Task<T>` or `ValueTask<T>`, where `T` is your result type, and then you can use the `return` keyword as though your method were a normal, non-async method, as [Example 17-12](#) shows.

Example 17-12. Returning a `Task<T>`

```
public static async Task<string?> GetServerHeaderAsync(
    string url, IHttpClientFactory cf)
{
    using (HttpClient w = cf.CreateClient())
    {
        var request = new HttpRequestMessage(HttpMethod.Head, url);
        HttpResponseMessage response = await w.SendAsync(
            request, HttpCompletionOption.ResponseHeadersRead);

        string? result = null;
        IEnumerable<string>? values;
```

```

        if (response.Headers.TryGetValue("Server", out values))
        {
            result = values.FirstOrDefault();
        }
        return result;
    }
}

```

This fetches HTTP headers asynchronously in the same way as [Example 17-1](#), but instead of displaying the results, this picks out the value of the first `Server:` header and makes that the result of the `Task<string?>` that this method returns. (It needs to be a nullable string because the header might not be present.) As you can see, the `return` statement just returns a `string?`, even though the method's return type is `Task<string?>`. The compiler generates code that completes the task and arranges for that string to be the result. With either a `Task` or `Task<T>` return type, the generated code produces a task similar to the kind you would get using `TaskCompletionSource<T>`, as described in [Chapter 16](#).

NOTE

Just as the `await` keyword can use any asynchronous method that fits a particular pattern (described later), C# offers the same flexibility when it comes to implementing an asynchronous method. You are not limited to `Task`, `Task<T>`, `ValueTask`, and `ValueTask<T>`. You can return any type that meets two conditions: it must be annotated with the `AsyncMethodBuilder` attribute, identifying a class that the compiler can use to manage the progress and completion of the task, and it must also offer a `GetAwaiter` method that returns a type implementing the `INotifyCompletion` interface.

There's very little downside to returning one of the built-in task types. Callers are not obliged to do anything with it, so your method will be just as easy to use as a `void` method but with the added advantage that a task is available to callers that want one. About the only reason for returning `void` would be if some external constraint forces your method to have a particular signature. For example, most event handlers are required to have a return type of `void`—that's why some of my earlier examples did

it. But unless you are forced to use it, `void` is not a recommended return type for an asynchronous method.

The program entry point (typically called `Main`) is a special case. The .NET runtime doesn't support asynchronous entry points, and it expects this method to return either `void` or `int`. Despite this, C# lets `Main` return either `Task` or `Task<int>`, in which case the C# compiler will generate a hidden method that acts as the real entry point. This calls your asynchronous `Main` and then blocks until the task it returns completes. This makes it possible for `Main` to be `async` (although the compiler will generate the wrapper when you use these return types even if you don't make the method `async`). If you use top-level statements, there will be no explicit declaration of `Main`, but if you use `await` in these statements, the compiler will choose a return type of `Task`, or, if you use a `return` statement to produce an exit code, `Task<int>`.

Applying `async` to Nested Methods

In the examples shown so far, I have applied the `async` keyword to ordinary methods. You can also use it on anonymous functions (either anonymous methods or lambdas) and local functions. For example, if you're writing a program that creates UI elements programmatically, you may find it convenient to attach event handlers written as lambdas, and you might want to make some of those asynchronous, as [Example 17-13](#) does.

Example 17-13. An asynchronous lambda

```
okButton.Click += async (s, e) =>
{
    using (HttpClient w = this.clientFactory.CreateClient())
    {
        infoTextBlock.Text = await w.GetStringAsync(uriTextBox.Text);
    }
};
```

The await Pattern

The majority of the asynchronous APIs that support the `await` keyword will return a TPL task of some kind. However, C# does not absolutely require this. It will `await` anything that implements a particular pattern. Moreover, although `Task` supports this pattern, the way it works means that the compiler uses tasks in a slightly different way than you would when using the TPL directly—this is partly why I said earlier that the code showing task-based asynchronous equivalents to `await`-based code did not represent exactly what the compiler does. In this section, I'm going to show how the compiler uses tasks and other types that support `await` to better illustrate how it really works.

I'll create a custom implementation of the `await` pattern to show what the C# compiler expects. [Example 17-14](#) shows an asynchronous method, `UseCustomAsync`, that uses this custom implementation. It assigns the result of the `await` expression into a `string`, so it clearly expects the asynchronous operation to produce a `string` as its output. It calls a method, `CustomAsync`, which returns our implementation of the pattern (which will be shown later in [Example 17-15](#)). As you can see, this is not a `Task<string>`.

Example 17-14. Calling a custom awaitable implementation

```
static async Task UseCustomAsync()
{
    string result = await CustomAsync();
    Console.WriteLine(result);
}

public static MyAwaitableType CustomAsync()
{
    return new MyAwaitableType();
}
```

The compiler expects the `await` keyword's operand to be a type that provides a method called `GetAwaiter`. This can be an ordinary instance member or an extension method. (So it is possible to make `await` work with a type that does not support it innately by defining a suitable exten-

sion method.) This method must return an object or value, known as an *awaiter*, that does three things.

First, the awaiter must provide a `bool` property called `IsCompleted`. The code that the compiler generates for the `await` uses this to discover whether the operation has already finished. In situations where no slow work needs to be done (e.g., when a call to `ReadAsync` on a `Stream` can be handled immediately with data that the stream already has in a buffer), it would be a waste to set up a callback. So `await` avoids creating an unnecessary delegate if the `IsCompleted` property returns `true`, and it will just continue straight on with the remainder of the method. (Passing `ConfigureAwaitOptions.ForceYielding` to `ConfigureAwait` defeats this optimization by supplying an awaiter where `IsCompleted` initially returns `false` even when the underlying task has in fact completed.)

The compiler also requires a way to get the result once the work is complete, so the awaiter must have a `GetResult` method. Its return type defines the result type of the operation—it will be the type of the `await` expression. (If there is no result, the return type is `void`. `GetResult` still needs to be present, because it is responsible for throwing exceptions if the operation fails.) Since [Example 17-14](#) assigns the result of the `await` into a variable of type `string`, the `GetResult` method of the awaiter returned by the `MyAwaitableType` class's `GetAwaiter` must be `string` (or some type implicitly convertible to `string`).

Finally, the compiler needs to be able to supply a callback. If `IsCompleted` returns `false`, indicating that the operation is not yet complete, the code generated for the `await` expression will create a delegate that will run the rest of the method. It needs to be able to pass that to the awaiter. (This is similar to passing a delegate to a task's `ContinueWith` method.) For this, the compiler requires not just a method but also an interface. You are required to implement `INotifyCompletion`, and there's an optional interface that it's recommended you also implement where possible called `ICriticalNotifyCompletion`. These do similar things: each defines a single method (`OnCompleted` and `UnsafeOnCompleted`, respectively) that takes a single `Action` delegate, and the awaiter must invoke this delegate once the operation completes. The distinction between these two interfaces and their corresponding methods is that the first requires the awaiter to flow the current execution context to the target

method, whereas the latter does not. The .NET runtime libraries features that the C# compiler uses to help build asynchronous methods always flow the execution context for you, so the generated code typically calls `UnsafeOnCompleted` where available to avoid flowing it twice. (If the compiler used `OnCompleted`, the awaiter would flow context too.) However, on .NET Framework, you'll find that security constraints may prevent the use of `UnsafeOnCompleted`. (.NET Framework had a concept of *untrusted code*. Code from potentially untrustworthy origins—perhaps because it was downloaded from the internet—would be subject to various constraints. This concept was dropped in .NET, but various vestiges remain, such as this design detail of asynchronous operations.) Because `UnsafeOnCompleted` does not flow execution context, untrusted code must not be allowed to call it, because that would provide a way to bypass certain security mechanisms. .NET Framework implementations of `UnsafeOnCompleted` provided for the various task types are marked with the `SecurityCriticalAttribute`, which means that only fully trusted code can call it. We need `OnCompleted` so that partially trusted code is able to use the awaiter.

[Example 17-15](#) shows the minimum viable implementation of the awaiter pattern. This is oversimplified, because it always completes synchronously, so its `OnCompleted` method doesn't do anything. If you use the `await` keyword on an instance of `MyAwaitableType`, the code that the C# compiler generates will never call `OnCompleted`. The `await` pattern requires that `OnCompleted` is only called if `IsCompleted` returns `false`, and, in this example, `IsCompleted` always returns `true`. This is why I've made `OnCompleted` throw an exception. However, although this example is unrealistically simple, it will serve to illustrate what `await` does.

Example 17-15. An excessively simple `await` pattern implementation

```
public class MyAwaitableType
{
    public MinimalAwaiter GetAwaiter()
    {
        return new MinimalAwaiter();
    }

    public class MinimalAwaiter : INotifyCompletion
```

```

{
    public bool IsCompleted => true;

    public string GetResult() => "This is a result";

    public void OnCompleted(Action continuation)
    {
        throw new NotImplementedException();
    }
}
}

```

With this code in place, we can see what [Example 17-14](#) will do. It will call `GetAwaiter` on the `MyAwaitableType` instance returned by the `CustomAsync` method. Then it will test the awaiter's `IsCompleted` property, and if it's `true` (which it will be), it will run the rest of the method immediately. The compiler doesn't know `IsCompleted` will always be `true` in this case, so it generates code to handle the `false` case. This will create a delegate that, when invoked, will run the rest of the method and pass that delegate to the waiter's `OnCompleted` method. (I've not provided `UnsafeOnCompleted` here, so it is forced to use `OnCompleted`.) [Example 17-16](#) shows code that does all of this.

Example 17-16. A very rough approximation of what `await` does

```

static void ManualUseCustomAsync()
{
    var awaiter = CustomAsync().GetAwaiter();
    if (awaiter.IsCompleted)
    {
        TheRest(awaiter);
    }
    else
    {
        awaiter.OnCompleted(() => TheRest(awaiter));
    }
}

private static void TheRest(MyAwaitableType.MinimalAwaiter awaiter)
{
    string result = awaiter.GetResult();
}

```

```
    Console.WriteLine(result);  
}
```

I've split the method into two pieces, because the C# compiler avoids creating a delegate in the case where `IsCompleted` is `true`, and I wanted to do the same. However, this is not quite what the C# compiler does—it also manages to avoid creating an extra method for each `await` statement, but this means it has to create considerably more complex code. In fact, for methods that just contain a single `await`, it introduces rather more overhead than [Example 17-16](#). However, once the number of `await` expressions starts to increase, the complexity pays off, because the compiler does not need to add any further methods. [Example 17-17](#) shows something closer to what the compiler does.

Example 17-17. A slightly closer approximation to how `await` works

```
private class ManualUseCustomAsyncState  
{  
    private int state;  
    private MyAwaitableType.MinimalAwaiter? awaiter;  
  
    public void MoveNext()  
    {  
        if (state == 0)  
        {  
            awaiter = CustomAsync().GetAwaiter();  
            if (!awaiter.IsCompleted)  
            {  
                state = 1;  
                awaiter.OnCompleted(MoveNext);  
                return;  
            }  
        }  
        string result = awaiter!.GetResult();  
        Console.WriteLine(result);  
    }  
}  
  
static void ManualUseCustomAsync()  
{  
    var s = new ManualUseCustomAsyncState();
```

```
s.MoveNext();  
}
```

This is still simpler than the real code, but it shows the basic strategy: the compiler generates a nested type that acts as a state machine. This has a field (`state`) that keeps track of where the method has got to so far, and it also contains fields corresponding to the method's local variables (just the `awaiter` variable in this example). When an asynchronous operation does not block (i.e., its `IsCompleted` returns `true` immediately), the method can just continue to the next part, but once it encounters an operation that needs some time, it updates the `state` variable to remember where it is and then uses the relevant `awaiter`'s `OnCompleted` method. Notice that the method it asks to be called on completion is the same one that is already running: `MoveNext`. And this continues to be the case no matter how many `await`s you need to perform—every completion callback invokes the same method; the class simply remembers how far it had already gotten, and the method picks up from there. That way, no matter how many times an `await` blocks, it never needs to create more than one delegate.

I won't show the real generated code. It is borderline unreadable, because it contains a lot of *unspeakable* identifiers. (Remember from [Chapter 3](#) that when the C# compiler needs to generate items with identifiers that must not collide with or be directly visible to our code, it creates a name that the runtime considers legal but that is not legal in C#; this is called an *unspeakable* name.) Moreover, the compiler-generated code uses various helper classes from the `System.Runtime.CompilerServices` namespace that are intended for use only from asynchronous methods to manage things like determining which of the completion interfaces the `awaiter` supports and handling the related execution context flow. Also, if the method returns a task, there are additional helpers to create and update that. But when it comes to understanding the nature of the relationship between an awaitable type and the code the compiler produces for an `await` expression, [Example 17-17](#) gives a fair impression.

Error Handling

The `await` keyword deals with exceptions much as you'd hope it would: if an asynchronous operation fails, the exception emerges from the `await` expression that was consuming that operation. The general principle that asynchronous code can be structured in the same way as ordinary synchronous code continues to apply in the face of exceptions, and the compiler does whatever work is required to make that possible.

[Example 17-18](#) contains two asynchronous operations, one of which occurs in a loop. This is similar to [Example 17-5](#). It does something a bit different with the content it fetches, but most importantly, it returns a task. This provides a place for an error to go if any of the operations should fail.

Example 17-18. Multiple potential points of failure

```
private static async Task<string> FindLongestLineAsync(
    string url, IHttpClientFactory cf)
{
    using (HttpClient w = cf.CreateClient())
    {
        Stream body = await w.GetStreamAsync(url);
        using (var bodyTextReader = new StreamReader(body))
        {
            string longestLine = string.Empty;
            while (!bodyTextReader.EndOfStream)
            {
                string? line = await bodyTextReader.ReadLineAsync();
                if (line is not null && line.Length > longestLine.Length)
                {
                    longestLine = line;
                }
            }
            return longestLine;
        }
    }
}
```

Exceptions are potentially challenging with asynchronous operations because by the time a failure occurs, the method call that originally started

the work is likely to have returned. The `FindLongestLineAsync` method in this example will usually return as soon as it executes the first `await` expression. (It's possible that it won't—if HTTP caching is in use, or if the `IHttpClientFactory` returns a client configured as a fake that never makes any real requests, this operation could succeed immediately. But typically, that operation will take some time, causing the method to return.) Suppose this operation succeeds and the rest of the method starts to run, but partway through the loop that retrieves the body of the response, the computer loses network connectivity. This will cause one of the operations started by `ReadLineAsync` to fail.

An exception will emerge from the `await` for that operation. There is no exception handling in this method, so what should happen next?

Normally, you'd expect the exception to start working its way up the stack, but what's above this method on the stack? It almost certainly won't be the code that originally called it—remember, the method will usually return as soon as it hits the first `await`, so at this stage, we're running as a result of being called back by the awaiter for the task returned by `ReadLineAsync`. Chances are, we'll be running on some thread from the thread pool, and the code directly above us in the stack will be part of the task awaiter. This won't know what to do with our exception.

But the exception does not propagate up the stack. When an exception goes unhandled in an `async` method that returns a task, the compiler-generated code catches it and puts the task returned by that method into a faulted state (which will in turn mean that anything that was waiting for that task can now continue). If the code that called

`FindLongestLineAsync` is working directly with the TPL, it will be able to see the exception by detecting that faulted state and retrieving the task's `Exception` property. Alternatively, it can either call `Wait` or fetch the task's `Result` property, and in either case, the task will throw an `AggregateException` containing the original exception. But if the code calling `FindLongestLineAsync` uses `await` on the task we return, the exception gets rethrown from that. From the calling code's point of view, it looks just like the exception emerged as it would normally, as

[Example 17-19](#) shows.

Example 17-19. Handling exceptions from `await`

```
try
{
    string longest = await FindLongestLineAsync(
        "http://192.168.22.1/", this.clientFactory);
    Console.WriteLine($"Longest line: {longest}");
}
catch (HttpRequestException ex)
{
    Console.WriteLine($"Error fetching page: {ex.Message}");
}
```

This is almost deceptively simple. Remember that the compiler performs substantial restructuring of the code around each `await`, and the execution of what looks like a single method may involve multiple calls in practice. So preserving the semantics of even a simple exception handling block like this (or related constructs, such as a `using` statement) is non-trivial. If you have ever attempted to write equivalent error handling for asynchronous work without the help of the compiler, you'll appreciate how much C# is doing for you here.

NOTE

The `await` does not rethrow the `AggregateException` provided by the task's `Exception` property. It rethrows the original exception. This enables `async` methods to handle the error in the same way synchronous code would.

Validating Arguments

There's one potentially surprising aspect of the way C# automatically reports exceptions through the task your asynchronous method returns. It means that code such as that in [Example 17-20](#) doesn't do what you might expect.

Example 17-20. Potentially surprising argument validation

```
public async Task<string> FindLongestLineAsync(string url)
{
```

```
ArgumentNullException.ThrowIfNull(url);
```

```
...
```

Inside an `async` method, the compiler treats all exceptions in the same way: none are allowed to pass up the stack as they would with a normal method, and they will always be reported by faulting the returned task. This is true even of exceptions thrown before the first `await`. In this example, the argument validation happens before the method does anything else, so at that stage, we will still be running on the original caller's thread. You might have thought that an argument exception thrown by this part of the code would propagate directly back to the caller. In fact, the caller will see a nonexceptional return, producing a task that is in a faulted state.

If the calling method immediately calls `await` on the return task, this won't matter much—it will see the exception in any case. But some code may choose not to wait immediately, in which case it won't see the argument exception until later. For simple argument validation exceptions where the caller has clearly made a programming error, you might expect code to throw an exception immediately, but this code doesn't do that.

NOTE

If it's not possible to determine whether a particular argument is valid without performing slow work, you will not be able to throw immediately if you want a truly asynchronous method. In that case, you would need to decide whether you would rather have the method block until it can validate all arguments or have argument exceptions be reported via the returned task instead of being thrown immediately.

Most `async` methods work this way, but suppose you want to throw this kind of exception straightaway (e.g., because it's being called from code that does not immediately `await` the result, and you'd like to discover the problem as soon as possible). The usual technique is to write a normal method that validates the arguments before calling an `async` method that does the work, and to make that second method either private or local. (You would have to do something similar to perform immediate argument validation with iterators too, incidentally. Iterators were described

in [Chapter 5](#).) [Example 17-21](#) shows such a public wrapper method and the start of the method it calls to do the real work.

Example 17-21. Validating arguments for `async` methods

```
public static Task<string> FindLongestLineAsync(string url)
{
    ArgumentNullException.ThrowIfNull(url);
    return FindLongestLineCore(url);

    static async Task<string> FindLongestLineCore(string url)
    {
        ...
    }
}
```

Because the public method is not marked with `async`, any exceptions it throws will propagate directly to the caller. But any failures that occur once the work is underway in the local method will be reported through the task.

I've chosen to forward the `url` argument to the local method. I didn't have to, because a local method can access its containing method's variables. However, relying on that causes the compiler to create a type to hold the locals to share them across the methods. Where possible, it will make this a value type, passing it by reference to the inner type, but in cases where the inner method's scope might outlive the outer method, it can't do that. And since the local method here is `async`, it is likely to continue to run long after the outer method's stack frame no longer exists, so this would cause the compiler to create a reference type just to hold that `url` argument. By passing the argument in, we avoid this (and I've marked the method as `static` to indicate that this is my intent—this means the compiler will produce an error if I inadvertently use anything from the outer method in the local one). The compiler will probably still have to generate code that creates an object to hold on to local variables in the inner method during asynchronous execution, but at least we've avoided creating more objects than necessary.

Singular and Multiple Exceptions

As [Chapter 16](#) showed, the TPL defines a model for reporting multiple errors—a task's `Exception` property returns an `AggregateException`. Even if there is only a single failure, you still have to extract it from its containing `AggregateException`. However, if you use the `await` keyword, it does this for you—as you saw in [Example 17-19](#), it retrieves the first exception in the `InnerExceptions` and rethrows that.

This is handy when the operation can produce only a single failure—it saves you from having to write additional code to handle the aggregate exception and then dig out the contents. (If you're using a task returned by an `async` method, it will never contain more than one exception.) However, it does present a problem if you're working with composite tasks that can fail in multiple ways simultaneously. For example, `Task.WhenAll` takes a collection of tasks and returns a single task that completes only when all its constituent tasks complete. If some of them complete by failing, you'll get an `AggregateException` that contains multiple errors. If you use `await` with such an operation, it will throw only the first of those exceptions back to you.

The usual TPL mechanisms—the `Wait` method or the `Result` property—provide the complete set of errors (by throwing the `AggregateException` itself instead of its first inner exception), but they both block the thread if the task is not yet complete. What if you want the efficient asynchronous operation of `await`, which uses threads only when there's something for them to do, but you still want to see all the errors? [Example 17-22](#) shows one approach.

Example 17-22. Throwless awaiting followed by `Wait`

```
static async Task CatchAll(Task[] ts)
{
    try
    {
        var t = Task.WhenAll(ts);
        await t.ConfigureAwait(ConfigAwaitOptions.SuppressThrowing);
        t.Wait();
    }
    catch (AggregateException all)
```

```

    {
        Console.WriteLine(all);
    }
}

```

This uses `await` to take advantage of the efficient nature of asynchronous C# methods, but it uses a new feature of .NET 8.0 to indicate that if the task faults, we don't want an exception to be thrown. The call to `Wait` will throw an `AggregateException` if anything failed, enabling the `catch` block to see all of the exceptions. And because we call `Wait` only after the `await` completes, we know the task is already finished, so the call will not block. (Alternatively, we could have not called `Wait` at all. After the `await` we could check `t.IsFaulted` to see if it failed, and get the exception with `t.Exception` if it did.)

Concurrent Operations and Missed Exceptions

The most straightforward way to use `await` is to do one thing after another, just as you would with synchronous code. Although doing work strictly sequentially may not sound like it takes full advantage of the potential of asynchronous code, it does make much more efficient use of the available threads than the synchronous equivalent, and it also works well in client-side UI code, leaving the UI thread free to respond to input even while work is then in progress. However, you might want to go further.

It is possible to kick off multiple pieces of work simultaneously. You can call an asynchronous API, and instead of using `await` immediately, you can store the result in a variable and then start another piece of work before waiting for both. Although this is a viable technique, and might reduce the overall execution time of your operations, there's a trap for the unwary, shown in [Example 17-23](#).

Example 17-23. How not to run multiple concurrent operations

```

static async Task GetSeveral(IHttpClientFactory cf)
{
    using (HttpClient w = cf.CreateClient())
    {
        w.MaxResponseContentBufferSize = 2_000_000;
    }
}

```

```

Task<string> g1 = w.GetStringAsync("https://endjin.com/");
Task<string> g2 = w.GetStringAsync("https://oreilly-com.ezproxy.sfpl.org");

// BAD!
Console.WriteLine((await g1).Length);
Console.WriteLine((await g2).Length);
}
}

```

This fetches content from two URLs concurrently. Having started both pieces of work, it uses two `await` expressions to collect the results of each and to display the lengths of the resulting strings. If the operations succeed, this will work, but it doesn't handle errors well. If the first operation fails, the code will never get as far as executing the second `await`. This means that if the second operation also fails, nothing will look at the exception it throws. Eventually, the TPL will detect that the exception has gone unobserved, which will result in the `UnobservedTaskException` event being raised. ([Chapter 16](#) discussed the TPL's unobserved exception handling.) The problem is that this will happen only very occasionally—it requires both operations to fail in quick succession—so it's something that would be very easy to miss in testing.

You could avoid this with careful exception handling—you could catch any exceptions that emerge from the first `await` before going on to execute the second, for example. Alternatively, you could use `Task.WhenAll` to wait for all the tasks as a single operation—this will produce a faulted task with an `AggregateException` if anything fails, enabling you to see all errors. Of course, as you saw in the preceding section, multiple failures of this kind are awkward to deal with when you're using `await`. But if you want to launch multiple asynchronous operations and have them all in flight simultaneously, you're going to need more complex code to coordinate the results than you would do when performing work sequentially. Even so, the `await` and `async` keywords still make life much easier.

Summary

Asynchronous operations do not block the thread from which they are invoked. This can make them more efficient than synchronous APIs, which

is particularly important on heavily loaded machines. It also makes them suitable for use on the client side, because they enable you to perform long-running work without causing the UI to become unresponsive. Without language support, asynchronous operations can be complex to use correctly, particularly when handling errors across multiple related operations. C#'s `await` keyword enables you to write asynchronous code in a style that looks just like normal synchronous code. It gets a little more complex if you want a single method to manage multiple concurrent operations, but even if you write an asynchronous method that does things strictly in order, you will get the benefits of making much more efficient use of threads in a server application—it will be able to support more simultaneous users, because each individual operation uses fewer resources—and on the client side, you'll get the benefit of a more responsive UI.

Methods that use `await` must be marked with the `async` keyword and should usually return one of `Task`, `Task<T>`, `ValueTask`, or `ValueTask<T>`. (C# allows a `void` return type, but you would normally use this only when you have no choice.) The compiler will arrange for this task to complete successfully once your method returns, or to complete with a fault if your method fails at any point in its execution. Because `await` can consume any `Task` or `Task<T>`, this makes it easy to split asynchronous logic across multiple methods, because a high-level method can `await` a lower-level `async` method. Usually, the work eventually ends up being performed by some task-based API, but it doesn't have to be, because `await` only demands a certain pattern—it will accept any expression on which you can invoke a `GetAwaiter` method to obtain a suitable type.

- ¹ This example is a bit contrived so that I can illustrate how `using` works in `async` methods. Disposing an `HttpClient` obtained from an `IHttpClientFactory` is normally optional, and in cases where you `new` up an `HttpClient` directly, it's better to hang on to it and reuse it, as discussed in [“Optional Disposal”](#).
- ² As it happens, [Example 17-3](#) does this too, because the TPL captures the execution context for us.
- ³ Strictly speaking, I should inspect the HTTP response headers to discover the encoding, and configure the `StreamReader` with that. Instead, I'm letting it detect

the encoding, which will work well enough for demonstration purposes.

- 4 These are available in .NET and .NET Standard 2.1. With .NET Framework, you will need to use the `Microsoft.Bcl.AsyncInterfaces` NuGet package.